UNIVERSIDAD POLITÉCNICA SALESIANA
CARRERA DE INGENIERÍA EN CIENCIAS DE LA COMPUTACIÓN

Análisis Comparativo de Robustez entre Momentos de Hu, Zernike y Shape Signature, y Reconocimiento de Hojas (Flavia) mediante Descriptores de Fourier de Coordenadas Complejas

Informe de Práctica 3-1 · 3-2 — Visión por Computador (G3)

Práctica de Laboratorio — Unidad 3

Autores:	Michael Lata (Lata Zambrano Michael Israel) Jordy Romero (Romero Armijos Jordy Jasmany)
Grupo:	G3 — Clases asignadas: 14, 15, 16, 17, 18, 19, 20
Docente:	Vladimir Robles Bykbaev
Asignatura:	Visión por Computador
Período:	Marzo – Agosto 2026
Fecha:	24 de junio de 2026

Resumen

En esta práctica desarrollamos dos sistemas de reconocimiento de formas basados en descriptores globales, sin usar aprendizaje profundo ni redes neuronales. En la **Parte 1** construimos un cuaderno en Python que compara la robustez de tres descriptores —**Momentos de Hu**, **Momentos de Zernike** y **Shape Signature**— frente a ruido sintético (Gaussiano y Sal & Pimienta en tres niveles) y rotaciones aleatorias de 0° a 360°, sobre el dataset *UPS-Writing-Skills* (círculo, triángulo, cuadrado). En la **Parte 2** implementamos una aplicación Android con una **librería nativa en C++** (vía JNI) que reconoce la especie de una hoja del *Flavia Dataset* usando el descriptor de Fourier de **coordenadas complejas** $s(n) = (x - x_c) + j(y - y_c)$.

Los resultados confirman la hipótesis de la guía: los Momentos de Hu se quedan en ∼64% de exactitud, mientras que Zernike y Shape Signature alcanzan ∼82% y ∼88% respectivamente, siendo claramente más robustos al ruido. La app móvil, validada en una tablet física, clasifica las 7 clases asignadas al grupo (14–20) con una exactitud del **93.5%** y muestra en pantalla la etiqueta de la especie junto a su descriptor de Fourier.

Índice

1 Introducción **3**

2 Parte 1: Robustez de Hu vs. Zernike vs. Shape Signature **3**
2.1 Dataset y preprocesamiento 3
2.2 Inyección de ruido y aumento de datos 4
2.3 Extracción de los tres descriptores 4
2.4 Clasificación y prevención de fuga de datos 5
2.5 Resultados 5

3 Parte 2: App Android — Reconocimiento de Hojas (Flavia) **5**
3.1 Arquitectura: un núcleo C++ compartido 7
3.2 Variante matemática: coordenadas complejas y FFT 7
3.3 Entrenamiento y clasificación 8
3.4 La aplicación funcionando en la tablet 8
3.5 Resultados: exactitud y formas confundidas 9

4 Conclusiones **9**

Introducción

El objetivo de esta práctica es evaluar y comparar *descriptores globales de forma*, que son funciones que resumen una figura completa en un vector de números (un "descriptor") invariante a transformaciones como rotación, escala o traslación. La idea central es que dos figuras de la misma clase deberían producir descriptores parecidos aunque estén rotadas o tengan ruido.

Trabajamos con dos enfoques complementarios. El primero son los **momentos** (Hu y Zernike), que integran toda la región de la figura ponderada por polinomios. El segundo es la **firma de la forma** (Shape Signature) y su variante de **coordenadas complejas**, que en lugar de mirar la región completa tratan el *contorno* como una señal y la analizan con la Transformada de Fourier.

Un punto importante de la práctica es que **no usamos redes neuronales ni aprendizaje profundo**. Todo se resuelve con matemática clásica de visión por computador (OpenCV, NumPy, SciPy, Scikit-Learn), de modo que cada paso es explicable y defendible con el material de la asignatura. La Parte 2 además exige llevar esa matemática a un **dispositivo móvil real**, lo que añade el reto de implementar los cálculos en C++ y conectarlos con la interfaz Android mediante JNI.

Parámetros personalizados del grupo

La práctica asignó parámetros personalizados que usamos de forma consistente en todo el proyecto:

- **Umbral de binarización** $T = 130$: separa la tinta del fondo en la segmentación.
- **Vecinos del clasificador** $K = 7$: número de vecinos del KNN.
- **Clases asignadas (G3):** 14, 15, 16, 17, 18, 19, 20 del Flavia Dataset.

Repositorios del Proyecto

- **Repositorio del proyecto (Parte 1 + Parte 2 + App Android):** `https://github.com/Whiteherobot/Practica3-VisionComputador-Hu-Zernike-Flavia`

Nota: el repositorio contiene el cuaderno de la Parte 1 (`notebooks/`), el núcleo C++ compartido (`cpp/`), la app Android (`mobile/`), los scripts de orquestación (`scripts/`) y los resultados (`results/`).

Parte 1: Robustez de Hu vs. Zernike vs. Shape Signature

Dataset y preprocesamiento

Usamos el dataset *UPS-Writing-Skills*, que contiene trazos hechos a mano de tres figuras geométricas: **círculo**, **triángulo** y **cuadrado**. Como son dibujos a mano alzada, los trazos a veces quedan abiertos o con huecos, así que la segmentación no puede ser un simple umbral.

Nuestra solución fue binarizar con el umbral $T = 130$ y luego tomar la **envolvente convexa** (*convex hull*) de la tinta significativa. Las tres clases son figuras convexas, así

que el *convex hull* reconstruye una silueta sólida y cerrada incluso cuando el trazo original tiene huecos. Esto evita que un dibujo abierto genere un contorno roto que arruine los descriptores.

```
# Binarizacion con el umbral personalizado T=130 + filtro de mediana
_, binaria = cv2.threshold(gris, T_THRESHOLD, 255, cv2.THRESH_BINARY_INV)
binaria = cv2.medianBlur(binaria, MEDIAN_KSIZE)  # robusto a Sal & Pimienta
# Envolvente convexa de la tinta significativa -> silueta solida
hull = cv2.convexHull(np.vstack(componentes_significativos))
```

El filtro de mediana antes de extraer los momentos es justamente la recomendación de la guía para mitigar el ruido Sal & Pimienta, que de otro modo distorsiona el cálculo del centroide.

Inyección de ruido y aumento de datos

Implementamos una función de inyección de ruido con dos tipos y tres niveles de intensidad (bajo, medio, alto), tal como pide la guía:

Ruido Gaussiano — se suma a cada píxel un valor de una distribución normal:

$$I_{\text{salida}}(x, y) = I(x, y) + \mathcal{N}(0,\, \sigma^2)$$

Ruido Sal & Pimienta — se ponen aleatoriamente píxeles en blanco (sal) o negro (pimienta) con probabilidad p (0.02, 0.06 y 0.12 para bajo, medio y alto).

Además, para cada imagen generamos varias **rotaciones aleatorias de 0° a 360°**. Esto es clave: si un descriptor es realmente invariante a la rotación, su exactitud no debería caer al introducir estas rotaciones.

Extracción de los tres descriptores

Momentos de Hu. Calculamos los 7 momentos invariantes de Hu y les aplicamos la transformación logarítmica vista en clase para que todos queden en órdenes de magnitud comparables:

$$h'_i = -\operatorname{sign}(h_i) \cdot \log_{10}(|h_i|)$$

Momentos de Zernike. Los implementamos *desde cero* (no hay función directa en OpenCV) con grado de polinomio $n = 8$. Se construye la base compleja $V_{nm}(\rho, \theta) = R_{nm}(\rho)\, e^{jm\theta}$ sobre el disco unitario y se proyecta la imagen sobre ella; usamos las **magnitudes** $|A_{nm}|$ porque son invariantes a la rotación (una rotación solo cambia la fase). Esto da 25 descriptores para $n = 8$.

Shape Signature. Calculamos la firma de distancia al centroide del contorno, le aplicamos la FFT y nos quedamos con los **13 primeros componentes de frecuencia**: el primero (DC) normaliza y los 12 siguientes forman el descriptor, exactamente como indica la guía.

Clasificación y prevención de fuga de datos

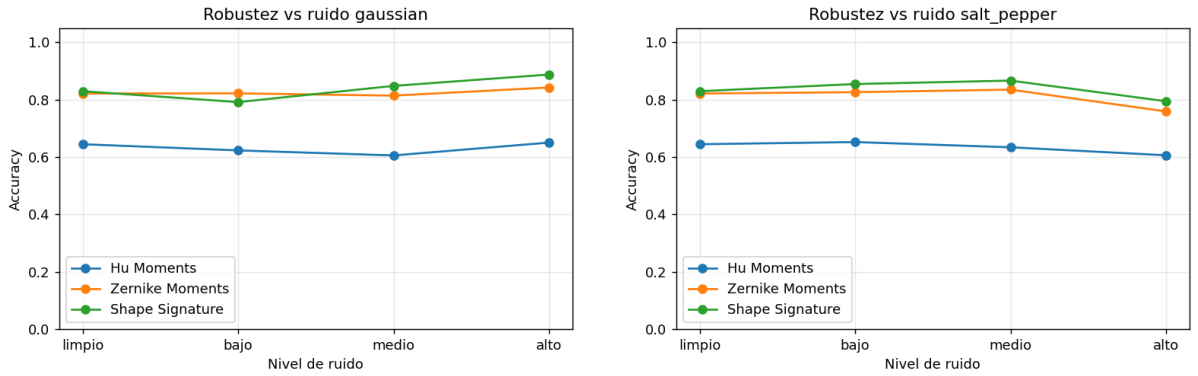
Entrenamos un clasificador **KNN con** $K = 7$ para cada descriptor. Un detalle metodológico importante: como generamos varias variantes (rotadas/ruidosas) de la *misma* imagen original, usamos `GroupShuffleSplit` para que todas las variantes de una imagen caigan juntas en *train* o en *test*, nunca repartidas. Si no se hace esto, el modelo "ve" en entrenamiento una versión casi idéntica de lo que luego evalúa, e infla artificialmente la exactitud (fuga de datos).

Resultados

La Tabla 1 resume la exactitud de cada descriptor por nivel de ruido, y las Figuras 1 y 2 lo muestran gráficamente.

Tabla 1: Exactitud (accuracy) por descriptor y nivel de ruido. KNN con $K = 7$.

Ruido	Descriptor	Limpio	Bajo	Medio	Alto
Gaussiano	Hu	0.645	0.623	0.606	0.651
Gaussiano	Zernike	0.822	0.822	0.814	0.842
Gaussiano	Shape Signature	0.830	0.792	0.848	**0.888**
Sal & Pimienta	Hu	0.645	0.652	0.634	0.606
Sal & Pimienta	Zernike	0.822	0.826	0.835	0.759
Sal & Pimienta	Shape Signature	0.830	0.855	**0.867**	0.795

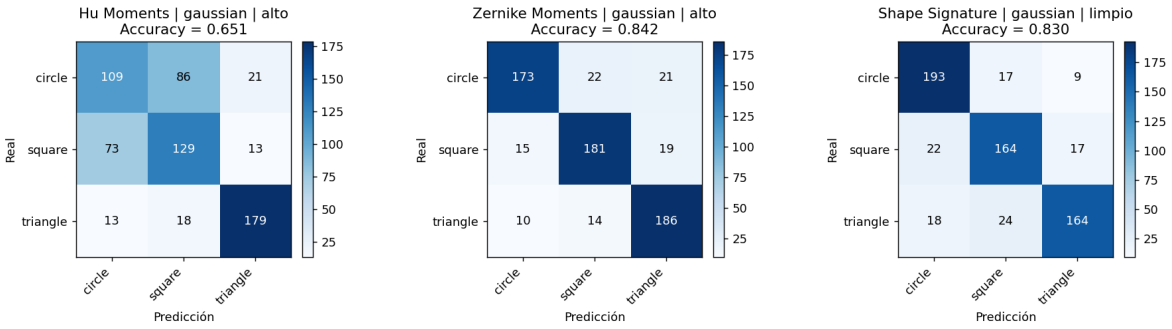


(a) Ruido Gaussiano (b) Ruido Sal & Pimienta

Figura 1: Curvas de robustez: exactitud frente al nivel de ruido. Hu (azul) se mantiene muy por debajo de Zernike (naranja) y Shape Signature (verde) en todos los niveles.

El resultado es claro y coincide con la conclusión esperada por la guía ("superioridad de la ortogonalidad"): los **Momentos de Hu** apenas superan el 60%, mientras que **Zernike** y **Shape Signature** se mantienen sobre el 80% incluso con ruido alto. La razón es que Hu solo tiene 7 valores muy redundantes entre sí, mientras que Zernike (base ortogonal) y la firma espectral capturan más detalle de la forma sin redundancia.

Parte 2: App Android — Reconocimiento de Hojas (Flavia)



(a) Hu — ruido alto (b) Zernike — ruido alto (c) Shape Sig. — limpio

Figura 2: Matrices de confusión por descriptor (se generó una por cada nivel de ruido). En Hu el triángulo y el cuadrado se confunden mucho; Zernike y Shape Signature separan mejor las tres clases.

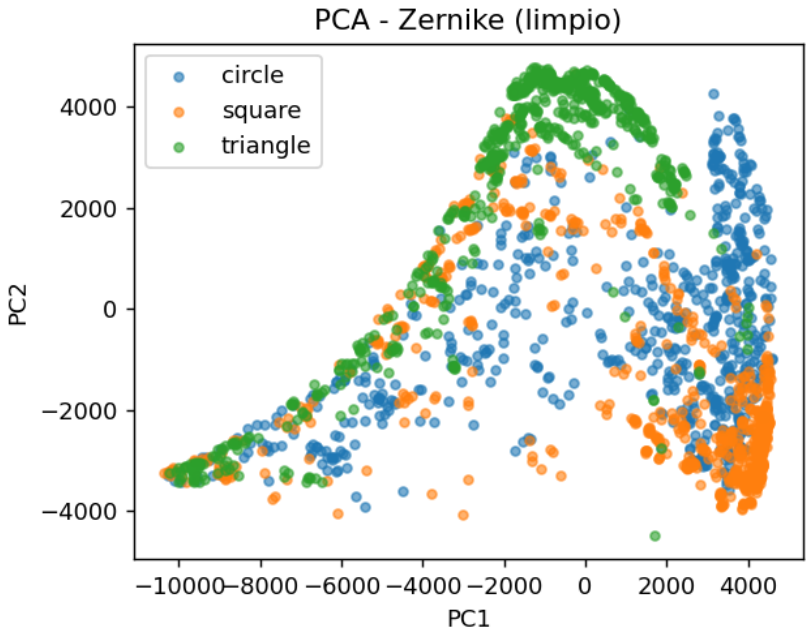


Figura 3: Proyección PCA de los descriptores de Zernike (datos limpios). Se observa cómo las tres clases forman grupos relativamente separados; al añadir ruido estos clusters se “mezclan”, lo que explica la caída de exactitud.

Arquitectura: un núcleo C++ compartido

La guía exige que "todos los cálculos se realicen en una librería nativa de C++". Para cumplirlo de forma limpia, escribimos el algoritmo **una sola vez** en C++ (`cpp/src/shape_descriptor.`
y lo compilamos en dos sitios: una CLI de escritorio (para validar y entrenar) y la app Android (vía JNI). Así garantizamos que el móvil y el PC producen *exactamente* el mismo descriptor.

```
UI (Java/Android)                                    Núcleo C++ (JNI)
─────────────────────────────────────────────────────────────────────
MainActivity -- botón --> NativeBridge.nativeClassify(matAddr)
DrawView (lienzo) --> OpenCV bitmapToMat --> native_lib.cpp
                    > shape_descriptor.cpp (cv::dft, |F|/|F(1)|, dist.
Euclídea, KNN K = 7)
```

Figura 4: Arquitectura de la app: la interfaz en Java captura el dibujo y delega todo el cálculo al núcleo C++ compartido mediante JNI.

Variante matemática: coordenadas complejas y FFT

En lugar de medir distancias al centroide, tratamos el contorno como una señal en el plano complejo, tal como pide la guía:

$$s(n) = \big(x(n) - x_c\big) + j\,\big(y(n) - y_c\big)$$

donde (x_c, y_c) es el centroide. El flujo en tiempo real es:

1. **Segmentación** por umbral adaptativo (el dibujo es trazo oscuro sobre fondo claro).
2. **Contorno** externo más grande con `cv::findContours`.
3. **DFT** de la señal compleja con `cv::dft` y `DFT_COMPLEX_OUTPUT`.
4. **Normalización** dividiendo por $|F(1)|$ (invarianza a escala) y descartando la fase (invarianza a rotación).

```cpp
// Senal compleja s(n) = (x - xc) + j (y - yc)  -> Mat N x 1, 2 canales
cv::Mat signal(nPoints, 1, CV_32FC2);
for (int i = 0; i < nPoints; ++i)
    signal.at<cv::Vec2f>(i, 0) =
        cv::Vec2f(pts[i].x - c.x, pts[i].y - c.y);

cv::Mat spectrum;
cv::dft(signal, spectrum, cv::DFT_COMPLEX_OUTPUT);  // FFT compleja

// Normalizar por |F(1)| y descartar fase (solo magnitudes)
float f1 = std::max(mag[1], mag[nPoints - 1]);
for (int k = 0; k < nDescriptors; ++k)
    desc.push_back(mag[k] / f1);
```

Un detalle no obvio que tuvimos que corregir. `cv::findContours` a veces devuelve el contorno en sentido *horario*, lo que coloca el armónico fundamental en $F(N-1)$ en lugar de $F(1)$. Cuando eso pasa, $|F(1)| \approx 0$ y al dividir por él el descriptor "explota" (valores enormes y distancias absurdas). La solución fue forzar la orientación **antihoraria**

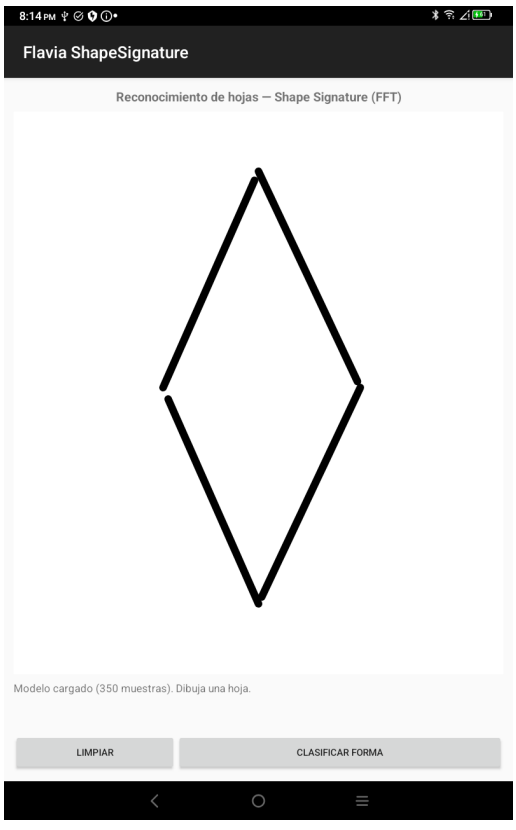
calculando el área con signo y, si es negativa, invertir el contorno. Tras esta corrección, el descriptor de una figura y el de su versión rotada/escalada quedan prácticamente idénticos (distancia < 0.001).

Entrenamiento y clasificación

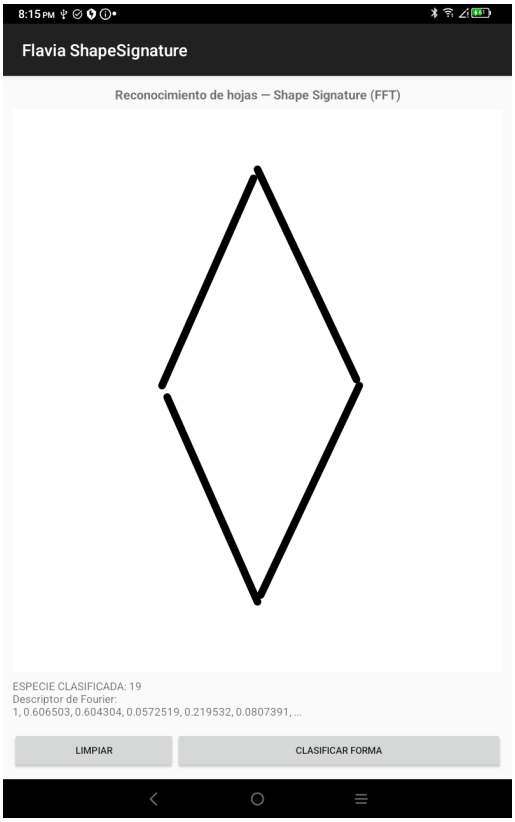
Dividimos el dataset en **80% train / 20% test por cada clase**. Para clasificar comparamos el descriptor de la imagen de prueba con los de entrenamiento usando **distancia Euclídea** (la misma métrica vista en el tema de Momentos de Hu) y un voto KNN con $K = 7$. Los descriptores de entrenamiento de las 7 clases (350 muestras) se empaquetan en `assets/model.csv` dentro del APK.

La aplicación funcionando en la tablet

La app presenta un área de dibujo y un botón "Clasificar forma". Al pulsarlo, toma el lienzo, lo convierte a `cv::Mat` y llama al núcleo C++, que devuelve la especie y los primeros valores del descriptor. La Figura 5 muestra la app ejecutándose en una tablet física (modelo 9185W, Android 15): se dibujó una hoja lanceolada y el sistema respondió **"ESPECIE CLASIFICADA: 19"** junto con su descriptor de Fourier, exactamente en el formato de la Ilustración 3 de la guía.



(a) Forma dibujada en el lienzo



(b) Resultado: "ESPECIE CLASIFICADA: 19"

Figura 5: App Android ejecutándose en dispositivo real. Tras pulsar "Clasificar forma", el núcleo C++ calcula el descriptor de Fourier de coordenadas complejas y devuelve la especie predicha con su vector descriptor.

Resultados: exactitud y formas confundidas

Sobre el conjunto de prueba (20% de cada clase), el sistema alcanza una **exactitud del 93.5%**, idéntica en la implementación de Python y la de C++ (lo que confirma que el núcleo compartido es consistente). La Figura 6 muestra la matriz de confusión y la Figura 7 algunos ejemplos de hojas que el sistema confunde.

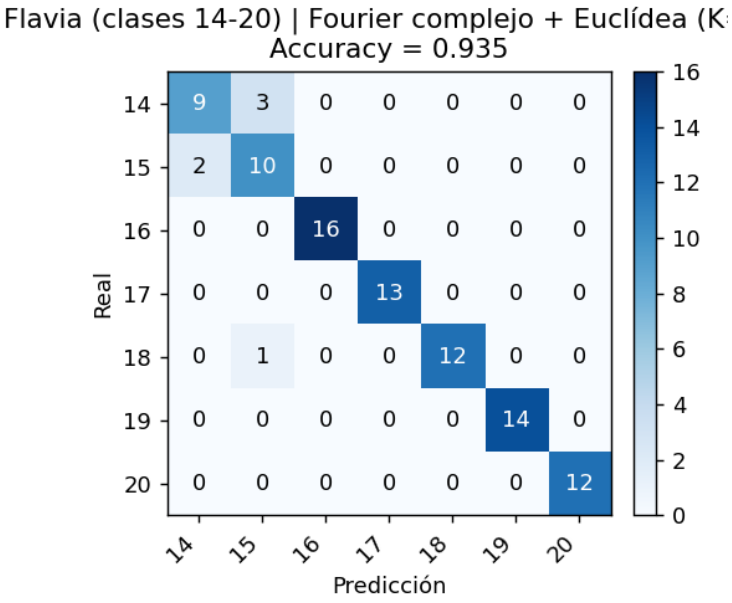


Figura 6: Matriz de confusión de las clases asignadas al G3 (14–20) sobre el conjunto de prueba. La diagonal concentra la mayoría de las predicciones (exactitud 93.5%).

Las confusiones tienen una explicación razonable: las clases 14, 15 y 18 corresponden a hojas de forma lanceolada (alargadas y de bordes lisos). Como el descriptor solo usa el *contorno* y descarta la textura interna y el color, dos hojas con siluetas casi iguales producen descriptores muy cercanos. Esto es coherente con la recomendación de la guía de explorar otras métricas de similitud (Manhattan, correlación) para separar clases parecidas.

Conclusiones

1. **La ortogonalidad sí importa.** Los Momentos de Hu se quedaron en ∼64% mientras que Zernike y Shape Signature superaron el 80% incluso con ruido alto. Hu solo aporta 7 valores muy correlacionados entre sí; las bases ortogonales y espectrales capturan más información sin redundancia, justo como anticipaba la guía.
2. **La normalización por $|F(1)|$ es frágil si el contorno está mal orientado.** El bug más difícil de la Parte 2 no fue la FFT en sí, sino que `findContours` devolvía contornos horarios que ponían el fundamental en $F(N-1)$. Forzar la orientación antihoraria fue lo que hizo que la invarianza a escala y rotación funcionara de verdad.
3. **Un solo núcleo C++ para PC y móvil ahorra muchos errores.** Compartir `shape_descriptor.cpp` entre la CLI y la app garantizó que la exactitud medida en el PC (93.5%) sea exactamente la que corre en la tablet, sin discrepancias por reimplementar el algoritmo dos veces.

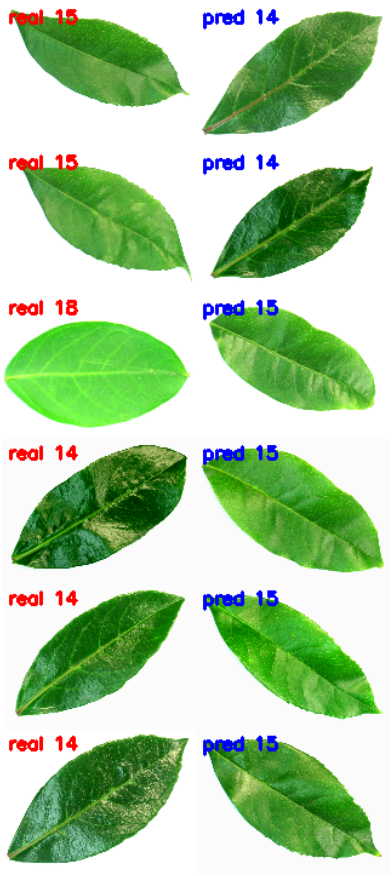


Figura 7: Ejemplos de hojas confundidas (real vs. predicha). Los errores se concentran entre las clases 14, 15 y 18, que son hojas **lanceoladas casi idénticas**: contornos alargados y simétricos cuya firma de Fourier es muy parecida.

4. **Llevar visión clásica a un móvil tiene fricción propia.** La matemática estaba lista, pero hacerla correr en la tablet exigió resolver tres problemas de integración: la imagen llega en 4 canales (RGBA), OpenCV exige STL compartida, y el lienzo de Android entrega bitmaps inmutables. Ninguno es "visión por computador", pero sin resolverlos la app no clasifica.
5. **El contorno no lo es todo.** El 6.5% de error se concentra en hojas lanceoladas casi idénticas (14/15/18). Un descriptor puramente de contorno no puede distinguirlas; haría falta incorporar textura o nervadura, lo cual queda fuera del alcance de esta práctica pero marca el límite natural del método.

Declaración de uso de herramientas de IA

Parte de este proyecto se desarrolló con asistencia del modelo de inteligencia artificial **Claude** (Anthropic, 2026). Para ser transparentes, detallamos a continuación los componentes *complicados* —los que no corresponden a una llamada directa de OpenCV ni están cubiertos paso a paso en las diapositivas— en los que se usó esta asistencia:

- **Implementación desde cero de los Momentos de Zernike** ($n = 8$): construcción de los polinomios radiales R_{nm}, la base compleja V_{nm} y la proyección A_{nm}, con cacheo de la base para acelerar el cálculo.
- **Corrección de orientación del contorno** (área con signo) para estabilizar la normalización por $|F(1)|$ del descriptor de Fourier de coordenadas complejas.
- **Puente JNI C++↔Java** (`native_lib.cpp`) y la configuración de CMake/Prefab para reutilizar el mismo núcleo C++ en escritorio y en Android.
- **Depuración en dispositivo real:** conversión de la imagen de 4 canales (RGBA) a gris (`COLOR_BGRA2GRAY`), uso de STL compartida (`c++_shared`) y copia mutable del bitmap del lienzo, que eran la causa de los cierres inesperados de la app.
- **Segmentación por envolvente convexa** de la tinta significativa para reconstruir siluetas sólidas a partir de trazos abiertos en la Parte 1.
- **Prevención de fuga de datos** con `GroupShuffleSplit` y la generación de las figuras y matrices de confusión.

La **teoría y la matemática** de la práctica (momentos de Hu, momentos de Zernike, firma de la forma, FFT, distancia Euclídea, KNN y PCA) se implementaron siguiendo directamente la *Guía de Práctica 3-1/3-2* del docente y el material teórico de la asignatura. La IA se utilizó como apoyo de implementación y depuración, no como sustituto del razonamiento sobre los descriptores, que entendemos y podemos defender.

Referencias

[1] Gonzalez, R.C., & Woods, R.E. (2018). *Digital Image Processing* (4th ed.). Pearson. *[Referencia principal del curso]*

[2] Robles Bykbaev, V. (2026). *Guía de Práctica de Laboratorio 3-1 / 3-2: Análisis comparativo de robustez entre Momentos de Hu y Zernike, y reconocimiento de for-*

mas mediante Firmas Espectrales con FFT de Coordenadas Complejas. Universidad Politécnica Salesiana.

[3] Hu, M.K. (1962). Visual pattern recognition by moment invariants. *IRE Transactions on Information Theory*, 8(2), 179–187.

[4] Teague, M.R. (1980). Image analysis via the general theory of moments. *Journal of the Optical Society of America*, 70(8), 920–930. *[Momentos de Zernike]*

[5] Zhang, D., & Lu, G. (2004). Review of shape representation and description techniques. *Pattern Recognition*, 37(1), 1–19. *[Shape Signature y descriptores de Fourier]*

[6] Wu, S.G., Bao, F.S., Xu, E.Y., Wang, Y.X., Chang, Y.F., & Xiang, Q.L. (2007). A leaf recognition algorithm for plant classification using probabilistic neural network. *IEEE ISSPIT*. *[Flavia Dataset]*

[7] Bradski, G., & Kaehler, A. (2008). *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media.

[8] Adolfo Gavilanes. (2024). *UPS-Writing-Skills Dataset*. Kaggle. https://www.kaggle.com/datasets/adolfogavilanes/ups-writing-skills

[9] Anthropic. (2026). *Claude — AI assistant*. https://www.anthropic.com. *[Asistencia en implementación y depuración de componentes C++/JNI/Android no cubiertos en el material de clase]*